Les Structures Répétitives en Algorithmique et Langage C : Boucles While, Do-While et For

Thème principal : Structures itératives (boucles while, do-while, for), prévention des boucles infinies et normes de codage C

Synthèse exécutive : Ce cours aborde de manière approfondie l'automatisation des traitements répétitifs en programmation algorithmique et en langage C. L'objectif est de supprimer toute duplication inutile de code conformément au principe DRY (Don't Repeat Yourself). Le cours détaille les trois boucles fondamentales : 'while' (à pré-condition), 'do-while' (à post-condition) et 'for' (à itération déterminée). Une attention particulière est portée à l'initialisation et à la mise à jour systématique des variables de contrôle pour éviter les boucles infinies, aux erreurs d'évaluation fréquentes en partiel (comme l'oubli du point-virgule final sur le do-while) ainsi qu'au respect strict de la norme de codage exigée à l'IUT d'Anglet (cartouche d'en-tête, tripartition, flèches de flot et style Allman).

1. Utilité des structures itératives et principe DRY

En programmation algorithmique, la répétition d'instructions ne doit jamais être gérée par du copier-coller. Dupliquer manuellement des lignes de code pour traiter 10 000 éléments d'un tableau ou redemander une saisie jusqu'à ce qu'elle soit correcte rend le code illisible, impossible à maintenir et extrêmement sujet aux bogues. Les structures itératives (ou boucles) permettent d'exécuter de façon automatisée et répétée un même bloc d'instructions tant qu'une condition logique (expression booléenne) reste vérifiée. Pour garantir qu'une boucle se termine, trois éléments indissociables sont requis : 1) une initialisation valide des variables de condition avant d'entrer dans la boucle, 2) une condition de maintien/sortie explicite, et 3) une modification effective de la variable de contrôle à chaque itération.

- Principe DRY (Don't Repeat Yourself) : Règle d'ingénierie logicielle interdisant la duplication manuelle de code pour répéter une tâche.
- Condition d'arrêt : Expression booléenne évaluée pour déterminer si la boucle doit continuer (Vrai) ou s'arrêter (Faux).
- Variable de contrôle / Compteur : Variable modifiée dans le corps de la boucle qui fait évoluer la condition booléenne vers sa valeur de sortie.
- Boucle infinie : Dysfonctionnement critique gelant le programme et saturant le processeur, provoqué par l'absence de mise à jour de la variable de contrôle.

Cas concret enseignant : *Le professeur compare la boucle au tapis roulant d'une usine d'emballage : tant qu'un colis est détecté par le capteur optique (condition vraie), le bras robotisé attrape le colis et l'emballe (instruction). Dès que le capteur ne détecte plus de colis ou que le quota fixé est atteint (condition fausse), le robot s'arrête. Sans capteur fonctionnel ou sans mise à jour du compteur de colis, le robot tournerait à vide indéfiniment jusqu'à surchauffer et détruire la machine.*

2. La boucle TANT QUE (while) : Pré-conditionnement

La structure 'while' est le choix algorithmique naturel lorsque le nombre exact d'itérations n'est pas connu à l'avance et dépend d'un événement dynamique externe (saisie utilisateur, fin de lecture d'un fichier, réception d'un signal réseau). En langage C, sa syntaxe est `while (condition) { ... }`. Comme l'évaluation s'effectue en amont, si la condition est fausse initialement, la boucle réalise exactement 0 itération. Deux erreurs classiques sont à éviter : l'omission d'initialisation de la variable de contrôle avant la boucle (lecture de mémoire indéterminée) et l'oubli d'incrémentation/mise à jour dans le corps du loop (boucle infinie).

- Pré-conditionnement : L'expression booléenne est évaluée AVANT l'accès au bloc d'instructions.
- Plage d'itérations (0 à N) : Si la condition est fausse dès la première entrée, le corps de la boucle n'est jamais exécuté.

• Initialisation préalable : Toute variable testée dans la condition du while doit obligatoirement posséder une valeur définie avant le test.

Cas concret enseignant : *Exemple du contrôle de code PIN au distributeur automatique de billets (DAB) : la machine demande le code secret TANT QUE le code saisi est incorrect ET QUE le nombre d'essais est strictement inférieur à 3. Si l'usager tape le bon code du premier coup, la condition devient fausse immédiatement et le corps de la boucle s'arrête.*

3. La boucle FAIRE... TANT QUE (do-while) : Post-conditionnement

À l'inverse de la boucle 'while', la structure 'do-while' garantit que les instructions internes s'exécutent une première fois avant de contrôler la condition logique de sortie. En langage C, sa syntaxe s'écrit `do { ... } while (condition);`. Elle est l'outil privilégié pour les contrôles de saisie (validation de bornes numériques) et les menus interactifs : il est en effet impossible de vérifier la validité d'une donnée saisie avant que l'utilisateur ne l'ait tapée au moins une fois au clavier. L'omission du point-virgule final constitue l'une des erreurs de syntaxe les plus lourdement sanctionnées à l'examen.

• Post-conditionnement : L'expression booléenne est évaluée APRÈS chaque exécution du bloc d'instructions.

• Garantie d'exécution (1 à N) : Le corps de la boucle s'exécute obligatoirement au moins une première fois.

• Syntaxe C stricte : Obligation absolue de placer un point-virgule ';' immédiatement après la parenthèse fermante du while.

Cas concret enseignant : *Cas du contrôle de saisie d'une note d'examen (entre 0 et 20) : 'On ne peut pas vérifier si la note entrée par un utilisateur est valide avant qu'il n'ait tapé au moins une valeur sur son clavier. On doit donc exécuter la saisie au moins une fois, ce qui fait du do-while la structure naturellement adaptée.'*

4. La boucle POUR (for) : Itération déterminée

La structure 'for' rationalise la mécanique itérative en réunissant les trois étapes clés sur une seule ligne d'en-tête. Le fonctionnement respecte un ordre strict : 1) Initialisation unique de la variable ; 2) Test de la condition ; 3) Si vrai, exécution du corps ; 4) Exécution de l'incrémentation/décrémentation ; 5) Retour à l'étape 2. Bien que le C autorise l'omission de certaines parties (ex: `for(;;)` qui crée une boucle infinie), la norme de l'IUT exige d'expliciter les trois expressions pour garantir une lisibilité optimale.

• En-tête condensé : Centralise l'initialisation, la condition de maintien et la mise à jour sous la forme `for(init; cond; inc)`.

• Nombre d'itérations déterminé : Structure recommandée lorsque le nombre exact de répétitions est connu avant d'entrer dans la boucle.

• Portée (scope) : Possibilité de limiter la durée de vie de la variable d'itération en la déclarant dans l'en-tête (standard C99).

Cas concret enseignant : *Métaphore des 5 tours de piste de l'athlète sur un stade : 'Avant de partir, vous savez que vous devez accomplir exactement 5 tours. Vous initialisez votre compteur au tour 1 (init), vous contrôlez que vous n'avez pas dépassé 5 (cond), et à chaque franchissement de la ligne d'arrivée, vous ajoutez 1 à votre compteur (incrémentation).'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou bloque indéfiniment quand j'oublie d'incrémenter la variable 'i' dans une boucle while ?
Réponse du professeur : Sans incrémentation de 'i', l'expression booléenne testée dans la condition du while conserve éternellement la même valeur (Vrai). Le programme entre dans une boucle infinie : le processeur exécute le même bloc en boucle saturant les ressources, et l'application cesse de répondre.

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle 'for' par une boucle 'while' ?
Réponse du professeur : Oui, de manière absolue. La boucle 'while' est la structure répétitive la plus générale. Un 'for(init; cond; inc)' se réécrit en plaçant 'init' avant le while, 'cond' en parenthèse du while, et 'inc' tout à la fin du bloc d'instructions. Cependant, on privilégie 'for' pour la lisibilité quand le nombre d'itérations est connu d'avance.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en langage C qui calcule la somme des N premiers entiers strictement positifs (1 + 2 + ... + N). Le programme doit obligatoirement sécuriser la saisie de N à l'aide d'une boucle 'do-while' afin de rejeter toute valeur inférieure ou égale à 0, puis calculer le cumul au moyen d'une boucle 'for'. Le code doit respecter la norme de codage de l'IUT d'Anglet (cartouche d'en-tête, tripartition CONSTANTES / VARIABLES / TRAITEMENTS, flèches de flot de données et style Allman).

Démarche de résolution :
Étape 1 : Déclarer les variables n, somme et i dans la section // VARIABLES. Étape 2 : Implémenter la saisie sécurisée via la boucle do-while avec la condition de maintien (n <= 0). La boucle force la re-saisie tant que l'utilisateur entre une valeur négative ou nulle. Étape 3 : Initialiser l'accumulateur 'somme' à 0 avant la boucle de calcul. Étape 4 : Exécuter la boucle for de i=1 jusqu'à i=n avec un pas de +1, en ajoutant la valeur de i à 'somme' à chaque tour. Étape 5 : Afficher le résultat final sur la sortie standard.

```
/*
  Fichier     : main.c
  Auteur      : Matéi
  Date        : 13/11/2026
  Rôle        : Saisie sécurisée de N (>0) et calcul de la somme 1..N
*/

#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    // CONSTANTES

    // VARIABLES
    int n;        // Nombre d'entiers à sommer
    int somme;    // Accumulateur de la somme
    int i;        // Compteur de boucle

    // TRAITEMENTS
    // (clavier) >> saisir >> n (avec validation n > 0)
    do
    {
        printf("Entrez un nombre entier N (>0) : ");
        scanf("%d", &n);
    }
    while (n <= 0);

    // Initialisation de l'accumulateur
    somme = 0;

    // Calcul de la somme des N premiers entiers
    for (i = 1; i <= n; i++)
    {
        somme += i;
    }

    // (ecran) << afficher << somme
    printf("La somme des %d premiers entiers est : %d\n", n, somme);

    return 0;
}
```

Résultat : Pour une saisie utilisateur N = 5 : - La boucle do-while accepte N = 5 car (5 <= 0) est FAUX. - La boucle for réalise 5 itérations : * i = 1 : somme = 0 + 1 = 1 * i = 2 : somme = 1 + 2 = 3 * i = 3 : somme = 3 + 3 = 6 * i = 4 : somme = 6 + 4 = 10 * i = 5 : somme = 10 + 5 = 15 Le programme affiche : 'La somme des 5 premiers entiers est : 15'. Si l'utilisateur avait saisi -3 au départ, le do-while aurait rejeté la valeur et réclamé la saisie.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale entre une boucle 'while' et une boucle 'do-while' concernant le nombre minimum d'itérations ?

R : Une boucle 'while' effectue de 0 à N itérations (le bloc peut ne jamais s'exécuter si la condition est fausse au départ), tandis qu'une boucle 'do-while' s'exécute au minimum 1 fois (1 à N itérations) car le test est effectué en post-condition.

Q : Que se passe-t-il si la condition initiale d'une boucle 'for' est fausse dès la première évaluation ?

R : La boucle est immédiatement sautée sans que les instructions de son corps ni l'expression d'incrémentation ne soient exécutées (0 itération).

Q : Quel oubli de syntaxe classique provoque une erreur de compilation sur la boucle 'do-while' en langage C ?

R : L'oubli du point-virgule final juste après la parenthèse fermante du while : `do { ... } while (condition);`.